

FRED TALKS

23rd May 2026

CONTENTS

Luddites and AI datacenters

SEANGOEDECKE.COM RSS FEED · 2789 WORDS

The Training Grounds: A Taxonomy of RL Environments for LLM Agents

HAN, NOT SOLO · 1766 WORDS

[Daily article] May 23: Running Out of Time (song)

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 400 WORDS

2.1.150

CHANGELOG · 6 WORDS

The Bag-of-Documents Model for Query Understanding and Retrieval

DANIEL TUNKELANG · 78 WORDS

Luddites and AI datacenters

SEANGOEDECKE.COM RSS FEED · 22 APR 2026 · [SOURCE](#)

Is it time to start burning down datacenters?

Some people think so. An Indianapolis city council member had his house recently shot up for supporting datacenters, and Sam Altman's home was firebombed (and then shot) shortly afterwards. People from all sides of the argument are sounding the alarm about imminent violence.

The obvious historical comparison is Luddism, the 19th-century phenomenon where English weavers and knitters destroyed the machines that were automating their work, and (in some cases) killed the machines' owners. Anti-AI people are reclaiming the term to describe themselves, and many of the leading lights of the anti-AI movement (like Brian Merchant or Gavin Mueller) have written books arguing more or less that the Luddites were right, and we ought to follow their example in order to resist AI automation¹.

Like many people, I have heard a lot about Luddism and Luddites, but only in the context of it being a general term for someone who is anti-technology. I was interested in learning more about the actual historical movement: what kind of people participated, what it was, and what it accomplished. I read Merchant's and Mueller's books, plus others², to try and figure all of this out. Who were the actual, historical Luddites? What can we learn from them about burning down datacenters?

Who were the Luddites?

The Luddites were a decentralized movement of artisans in the 1810s who engaged in violent protest - smashing machines, threatening violence, and ultimately killing people - over the fact that their jobs were being automated away. They were not rich, but they were certainly not unskilled labor: these were people who had apprenticed for seven years. They were mostly working from home, producing cloth from raw material given to them by their employer, often with tools rented from that same employer. They were working short weeks (three days, per William Gardiner) at their own discretion.

In the early 1800s, their skilled labor was becoming unnecessary. With the help of expensive machines, unskilled labor could now produce lower-quality cloth, so employers were beginning to pass over these artisans in favor of cheaper employees: children, unapprenticed workers, and

women³. Combined with the bad economic position of England at the time (at war with France, and thus deliberately cutting off much European trade), times were beginning to be very tough indeed. Starvation was a real threat.

What did they do?

Cloth artisans were groups of capable men who were used to getting their own way, knew each other very well, and were broadly respected in their communities. It was thus a natural response for them to organize into what was effectively a militant union. The Luddites would send anonymous threatening letters to their old (or current) bosses, warning them to stop using their machines. If they didn't comply, they would raid the workshop or factory, smashing the machines up.

They typically did not harm people, though they certainly delivered threats of bodily harm or even murder, and the raids were violent enough (e.g. shooting through windows) to have risked accidental deaths. In at least two instances where a factory owner was seen as unusually cruel, the Luddites did attempt assassinations: one unsuccessful, and one successful one that eventually prompted a crackdown that ended the movement for good.

Luddism was fully decentralized. Different communities could and did decide to engage in machine-raiding independently, particularly when news spread of the tactic succeeding. Although each community had its own influential men, there was never a single "leader of Luddism". King Ludd himself was a folk-tale figure. This made it an absolute nightmare for the British government to try and suppress them: putting down one Luddist group did nothing to prevent other groups from continuing to operate.

All the king's spies

I was surprised by how *difficult* it was for the government to get a hold of any of the local Luddist ringleaders. The government was willing to offer huge rewards to informers: at one point up to 40x the yearly wage. However, there were no takers for several years. Armies of spies were recruited and tasked with infiltrating Luddist groups, with absolutely no success.

Why was it so hard? Firstly, because the working class was so overwhelmingly pro-Luddist. People universally blamed the economic situation on the government and the factory owners (rightfully so, since the government had chosen to go to war and the factory owners had chosen to embrace automation). Secondly, the communities in question were so insular and tightly-knit that informers would have to rat on their friends and relatives. The handful of people who did eventually inform lived out the rest of their lives as pariahs.

Because each group was so insular, any spies trying to infiltrate the movement would have been complete strangers to the community, and would thus have a very hard time gaining the trust of a group of men who had known each other for their whole lives. The spies that did exist were restricted to the occasional inter-group Luddist meetings, where people didn't all know each other so closely. But it's unclear how important those meetings were, since Luddist groups didn't need to coordinate to achieve their goals. According to Merchant, the spies spent much of their time embellishing tales of an imminent revolution to encourage their employers to keep the money flowing.

The crackdown

In the absence of reliable information, the British government was forced to use force. And they did, sending 12,000 troops⁴ into the northern counties. This served mainly as an intimidation tactic, since there was no standing Luddite army to fight, and the soldiers spent most of their time marching back and forth or being abused by the townspeople.

More successful was the imposition of a full police state in Yorkshire, under the magistrate Joseph Radcliffe, who was empowered to randomly grab people off the street and interrogate them for days. That pressure eventually convinced a handful of people to give up their local Luddist organizers, who were tried and inevitably hanged. Their deaths (and the ensuing climate of fear) ended the high-water mark of Luddist activity. Even then, Luddist raids continued on and off for *six more years* before petering out.

Did the Luddites succeed?

This is a tricky question. In one sense the answer is obviously no: the movement was crushed, many of their leaders were executed, the textile industry continued to be automated, and today there are no longer thousands of jobs for skilled British weavers, knitters, spinners and dyers. The pro-automation side won.

However, they did achieve a number of short-lived victories. Their early threats often succeeded in preventing the building of a factory in a particular location, or in delaying the adoption of industrial machinery in a particular shop by years. In one case, hosiers that had been spooked by Luddite activity gave out pre-emptive bonuses to their workers to discourage them from smashing up their machines (which were indeed not smashed).

The Luddites also scared the hell out of the British government, who (encouraged by their over-eager spies) thought they might have a genuine revolution on their hands. While they didn't get many legal concessions at the time, the specter of Luddism must have loomed over the labor reform movement of the 1800s, which saw the first anti-child-labor laws and the beginnings of independent inspection of factories.

Finally, every book I read argued that the Luddism movement may have created the first idea of a "working class", by unifying many previously-independent groups of workers against a common enemy. Seen this way, the "political arm"⁵ of Luddism can arguably claim partial credit for every labor victory since the 1800s (though the ringleaders were still hanged and the weavers did still lose their jobs).

The Luddist approach in a nutshell

We can now describe the "Luddist approach" to fighting technological change:

1. Find a few conspirators in your existing community who agree with your political project (but don't join a broader organization, since that leaves you vulnerable)
2. Make public anonymous demands in support of your specific goals, backed up by threats of violence, signed by a fictional character that's easy for other groups to appropriate
3. If your threats are ignored, attack the physical machines in the dead of night, destroying them and threatening (but not killing) any guards
4. Hope your example inspires many more people to independently do (1)-(3) themselves
5. Keep raiding, optionally escalating to assassination of some of the bosses, until you bait a totalitarian crackdown from the government
6. Eventually get arrested and executed, to great public dismay
7. Twenty years later, your example inspires the first national trade unions

Note that starting or joining a national movement is *not* the Luddist approach. Staying almost entirely isolated in small cells helped the Luddists avoid government spies and made them impossible to root out without enforcing a police state. Note also that you need a *lot* of public support for this to work: so that you get a lot of copycat groups without having to explicitly organize them, and so that your property destruction and murder is taken sympathetically instead of getting you immediately reported and arrested.

Why Luddism is not a good model for the anti-AI movement

There are many reasons why this doesn't map onto the current anti-AI movement. First, Luddism grew from a homogeneous group of high-status workers whose jobs almost vanished overnight, not a broad group of people whose jobs are getting slightly worse because of AI (like the gig-economy workers Merchant endlessly references). That meant that Luddites had *really specific* asks: higher wages for piecework, a phased introduction of specific textiles machinery, and so on. They were not generally demanding that the machines all be immediately destroyed⁶.

Second, Luddism was very local. A pre-existing group of artisans in a particular town would gather in that town - either at work or an inn, say - and decide to petition or raid the businesses in that town that were harming their livelihoods. AI concerns are not like this. It isn't businesses in Chicago or Tokyo that are making decisions that imperil Chicago's or Tokyo's jobs, it's businesses in San Francisco. Unlike the Luddists, anti-AI activists can't naturally organize with people they already know to take direct action where they already live.

Third, Luddist *victory* could also be local. If you successfully lobby your local cloth business to not use a weaving machine, you have secured your job at that business for a while. But if you successfully lobby your town (or even your country!) to not build a datacenter, it doesn't meaningfully improve your local position, since your job can be as easily replaced by a datacenter on the other side of the planet.

A total failure of leadership

Reading through the history of the Luddites from a modern perspective, I was struck by the near-total absence of *good government*. The artisans were left to work out their grievances with their bosses more or less by themselves, with no formal channels for complaint or any attempt at mediation. When the government did intervene - in response to near-universal unrest in *half of the country* - they did this:

1. Make machine-breaking and oath-taking capital crimes
2. Dump thousands of soldiers more or less at random into the area, with no plan to guard factories or do anything beyond just hang around in case a revolution broke out
3. Empower a single magistrate to arrest and interrogate whoever he wanted in order to root out the conspiracy

I suppose it worked, in the sense that it eventually succeeded in stopping the Luddist raids. But I can't help but think that even a token gesture of compromise (say, requiring employers to make their wages public, or restricting the most cheap-and-nasty factory-made textile products) would have gone a long way towards calming things down. This almost actually happened! The 1812 Framework Knitters' Bill, which had these provisions in it, passed the House of Commons but was shot down in the House of Lords.

Why did the government fail to even make a token attempt at compromise? Before the industrial revolution, I wonder if the workers and bosses of the English textiles industry were genuinely able to often just work out their problems together, so the government never really needed to do large-scale mediation. When that changed - when automation first made it possible for the bosses to durably "win" - government took a long time to realize, so there were some unpleasant decades of disempowered workers trying to bully factory-owners (via riots and death threats), and factory-owners trying to brutalize workers (via direct violence and automation).

Final thoughts

The Luddites weren't anti-technology in general. Instead, they were against four or five specific machines that were automating away their skilled work. Contrary to many of the books I read, I think that's actually fairly well understood today. But what surprised me was how truly decentralized and widespread the movement was. Every town with weavers faced the same incentives (the bosses to industrialize, and the workers to fight back) which created Luddism locally. And *nobody* was willing to report the Luddites: not for years, or for what would have been a fortune in cash, or in disagreement with their actions. They were only stopped by a truly fascist crackdown, with the army in the streets and the secret police pulling away random citizens for arrest and questioning.

I can see why modern "Luddites" - who are genuinely anti-technology - talk so much about the legacy of original Luddites. Luddism was a grassroots organization which notched up some real short-term wins, enjoyed near-total support among the public, and didn't seem to be troubled by infighting at all⁷. If you're an anti-AI campaigner, I bet all of that sounds great⁸. But I'm not convinced that the neo-Luddites really are the inheritors of Luddism. A load-bearing feature of Luddism is that it was *local*: it didn't have manifestos, or leaders, or factions, or even much explicit ideology beyond the artisans' immediate practical concerns. These were local men striking back against the local factories harming their local jobs. That simply isn't the case with AI, where a datacenter in China can take my job in Australia.

If it isn't time to start burning down datacenters, what can we do? Well, workers today are better off than the Luddites were, because we stand on their shoulders. Unlike the Luddites, today's workers can all vote (and I suspect there will be no shortage of anti-AI candidates to vote for).

It's a much less exciting answer to say "try and get better laws passed" than to say "viva the revolution, where's my Molotov cocktail". Still, if the Luddites had that option, I suspect they would have used it.

1. I got linked [this article](#) calling AI a "fascist artifact" (on a blog called "Breaking Frames", a clear reference to Luddism) while I was writing this blog post.

↩

2. I really enjoyed Merchant's book and did not enjoy Mueller's (which I found to be 10% about the Luddites and 90% about interminable intra-Marxist ideological arguments). I also read [The Luddites](#), which was effectively a dry summary of the ground Merchant covers, a bunch of other essays, and went back and forth with ChatGPT and Claude on some of the key questions.

↩

3. Merchant (around page 134) attempts to characterize Luddism as a pro-feminist movement, citing some examples of women helping organize raids, but later on even he (page 162) quotes a representative of the Irish weaver's guild effectively saying "we don't have your English problems of women working in the industry". In general it's a bit frustrating that the popular books on Luddism are all fairly uncritically pro-Luddist (though not surprising, I suppose). Merchant doesn't touch at all on the Luddist [practice](#) of going around to knitting-shops with women and "discharging them from working".

↩

4. Sometimes this is described as more troops that were sent to fight Napoleon (even by Merchant himself on page 89), but that [isn't right](#).

↩

5. In quotes because it was not an official Luddist group (there were none), just people who were trying to stop the violence through lobbying and legislation.

↩

6. Otherwise why would any boss agree, instead of just waiting for the Luddites to do it themselves?

↩

7. As far as I can tell this is true: the Luddists basically had no internal conflict. I think this is because each individual cell knew each other well already, and so handled their disagreements privately (instead of by writing pamphlets), and disagreements between cells didn't matter that much because they had no need to coordinate.

↩

8. It beats the hell of the other popular reference, Dune's Butlerian Jihad, which was two generations of brutal violence followed by the reimposition of the feudal system. (Although, at least the Butlerian Jihad *succeeded*...)

↩

The Training Grounds: A Taxonomy of RL Environments for LLM Agents

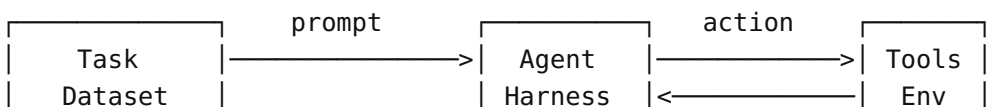
HAN, NOT SOLO · 22 MAR 2026 · [SOURCE](#)

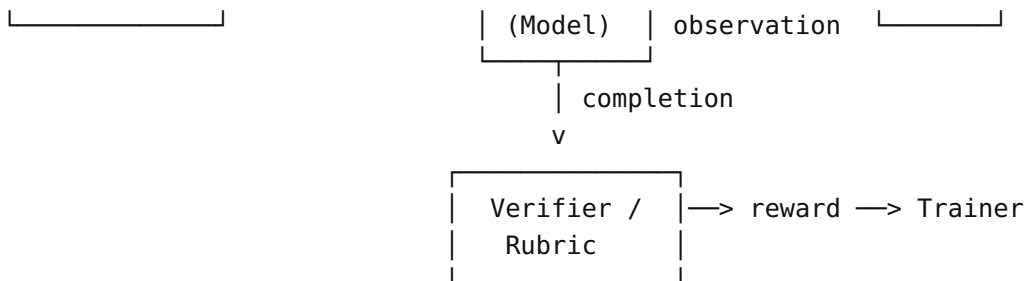
Model architecture gets all the attention. Post-training recipes follow close behind. The training environment — what the model actually practices on, how its work gets judged, what tools it can use — barely enters the conversation. That's the part that actually determines what the agent can learn to do.

A model trained only on single-turn Q&A will struggle the moment you ask it to maintain state across a 50-step enterprise workflow. A model trained with a poorly designed reward function will learn to game the metric, not solve the problem. The environment isn't a detail. It's half the system.

The Canonical Loop

An RL environment for an LLM agent bundles three things: a dataset of task inputs, a harness for the model, and a reward function to score outputs. The training loop looks like this:





Formally, a complete RL environment is a tuple:

Task distribution, harness, verifier, state management, configuration. Let's go through each.

The task distribution is the set of problems the agent trains on. Not all tasks are equal, and not just in difficulty. They vary structurally in ways that demand different capabilities:

Task Type	What the Agent Must Do	Example Systems
Single-turn Q&A	One prompt → one response, check answer	Math benchmarks, SimpleQA
Multi-hop search	Chain searches, synthesize sources	BrowseComp, WebWalkerQA
Open-ended research	No single correct answer; report quality matters	ADR-Bench, ResearchRubrics
Agentic tool-use	Call tools correctly in sequence	tau-bench, function-calling benchmarks
Stateful enterprise	Modify persistent DB state, work within access controls	EnterpriseOps-Gym
Code execution	Write code, run it, check outputs	SWE-Bench, LiveCodeBench

Training only on tasks with ground-truth answers produces an agent that's never learned to reason under ambiguity. Training only in clean, deterministic environments produces an agent that falls apart in production. The task distribution is a design decision with direct consequences.

Task synthesis is increasingly a first-class problem. With real-world research tasks, you rarely have a large labeled dataset. Strategies that have emerged:

- **Back translation:** Start from a desired output, reconstruct the query that would produce it
- **Graph-based synthesis:** Build a knowledge graph, generate multi-hop queries over it

- **Automated environment generation:** Use LLM coding agents to write new environment code. AutoEnv reports ~\$4/env average cost.
- **Curriculum construction:** Order tasks by difficulty and increase complexity during training

The cheapest-to-collect tasks are single-turn with verifiable answers. The most valuable tasks for long-horizon behavior are expensive to construct. This tension drives most environment design decisions.

The harness is the scaffolding that mediates between the model and the environment. It controls how the model interacts, not what it knows.

```
H = {
    rollout_protocol,    # SingleTurn | MultiTurn | Agentic
    tools,               # Available tools in this episode
    system_prompt,       # Instructions for the agent
    context_manager,     # How to handle context overflow
    turn_limit,          # Max interactions per episode
    sandbox,             # Code execution sandbox
    state                # Persistent state across turns
}
```

Rollout protocols range from trivial to complex:

Harness Type	Description	When to Use
Single-Turn	One prompt, one response	Math, factual QA
Multi-Turn	Back-and-forth dialogue	Games, structured tasks
Tool-Use	Model calls tools, receives results	Agent benchmarks
Stateful Tool-Use	Tools modify persistent state	Enterprise workflows, SWE-Bench
Agentic ReAct	Full Thought → Action → Observation loop	Deep research, complex workflows

Tools span a wide taxonomy:

Category	Tools	Deterministic?	Stateful?
Information retrieval	web_search, scholar_search	No (live web)	No
Content extraction	jina_reader, visit, web_scrape	No	No
Code execution	python_interpreter, shell, sandbox	Yes (given same code)	Yes

File operations	file_read, file_write	Yes	Yes
Browser automation	playwright, link_click	No	Yes
Task management	todo, section_write	Yes	Yes

The mix of deterministic/non-deterministic and stateful/stateless tools has real implications for reproducibility and reward assignment. Non-deterministic tools mean two runs of the same trajectory can produce different outcomes — which complicates both debugging and verifier design.

Context management is where most teams underinvest, especially for long-horizon tasks. A 600-turn research episode blows past any practical context window. Strategies used in production:

Strategy	Description	Trade-off
Recency-based retention	Keep N most recent turns	Simple, but loses early context
Markovian reconstruction	Reconstruct state from scratch each turn	Principled, expensive
Reference-preserving summarization	Summarize old context, keep citations	Preserves verifiability
Reference-preserving folding	Compress context without losing references	Best for research tasks

An agent doing multi-hour research needs to remember why it started searching in a particular direction twelve tool calls ago. Dropping that context causes repeated work and lost threads.

The verifier maps a completion to a reward:

In Atari, the score is unambiguous. In deep research, what counts as a good answer is not. The verifier is where this gets hard.

Type	Reward Signal	When to Use
Exact match	Binary (0/1)	Ground truth available
Code execution	Binary or partial	Output can be tested programmatically
LLM-as-judge	Continuous [0,1]	Open-ended quality, no other option
Checklist-style	Continuous	Multi-criteria research tasks
Evolving rubric (RLER)	Continuous	Resistant to reward hacking
Process reward model (PRM)	Per-step continuous	Long-horizon credit assignment

Pairwise comparison	Relative rank	Relative quality matters more than absolute
Multi-criteria composite	Weighted sum	Multiple quality dimensions

A few principles that actually matter in practice:

Verifiable beats judgeable. Programmatic checks — code execution, string match — are faster, cheaper, and more consistent than LLM-as-judge. Use LLM-as-judge when there’s no other option, not as the default.

Reward granularity is a separate decision from reward type. You can score at the trajectory level (did the final output pass?), turn level (was each tool invocation useful?), or per-step with process rewards. Turn-level supervision, as Nanbeige4.1 does across up to 600 tool calls, enables finer credit assignment — the model can learn that the problem was a bad search query in turn 23, not that the entire episode failed.

Static rubrics get gamed. Models learn to write answers that score well on your rubric rather than solving the problem. DR Tulu’s RLER (Rubric-Level Evolving Reward) co-evolves the rubric with the policy during training. Harder to exploit a moving target.

Noise injection is underrated. Step-DeepResearch deliberately injects 5–10% tool errors during training. The resulting model handles flaky APIs and unexpected failures in production significantly better.

State management determines whether the environment is stateless or persistent. Most academic benchmarks are stateless — each episode starts fresh. Enterprise environments are not. [EnterpriseOps-Gym](#) maintains 164 database tables and 512 tools across episodes. Actions in one task affect the state seen by subsequent tasks. That’s a fundamentally different problem for agents to solve.

Configuration covers turn limits, context budgets, sampling temperature, and curriculum scheduling. These are not afterthoughts. A turn limit of 5 vs. 600 changes what skills the agent can develop. [AgentScaler](#) uses a two-phase curriculum — fundamental capabilities first, then domain-specific tasks — and the ordering matters. Step-DeepResearch progressively scales context windows from 32K to 128K during mid-training.

The most consequential architectural question: where does the model sit relative to the environment?

Option A: Model outside the environment (decoupled). Model is served via API; the environment calls it at each step. Clean separation. Easy to swap models.

Option B: Model inside the environment (co-located). Model and environment share the same training loop. Lower latency, tighter integration, harder to reuse.

Option C: Split architecture. Trainer, model inference server, and environment are three separate processes communicating via API. This is where the field is landing.

How real systems implement this:

System	Topology	Notes
Prime Intellect verifiers	Option C (Split)	Env is a standalone Python package distributed via Environments Hub
Tongyi DeepResearch	Option B (Co-located)	Tools, context manager, verifiers inside the training pipeline
Step-DeepResearch	Option B (Co-located)	Single-agent ReAct loop embedded in training
MiroThinker	Option C (Split)	Tool servers and sandbox run independently from model
Tinker API	Option A (Decoupled, cloud)	Model stays remote; researcher sends <code>forward_backward</code> + <code>sample</code> calls via API
AutoEnv	Option A (Decoupled)	CoreEnv/ObsEnv/SkinEnv abstraction layers
EnterpriseOps-Gym	Option A (Decoupled)	Containerized sandbox accessible via any model API

The trade-offs:

Dimension	Model Outside (A/C)	Model Inside (B)
Flexibility	Swap models easily; env is reusable	Tighter integration
Scalability	Scale inference and training independently	Must scale everything together
Portability	Env packages are shareable	Env tied to training framework
Latency	Network overhead per tool call	No network overhead
RL compatibility	Works with any RL trainer	Usually tied to one trainer

The field has converged on Option C for production training. Prime Intellect’s architecture — environments as standalone installable packages that communicate with models via OpenAI-compatible API endpoints — is becoming the standard. The payoff: environments are publishable, trainer-agnostic, and inference and environment execution can be parallelized across nodes.

Tinker pushes this further by making even the training compute remote. The researcher controls the algorithm and never touches model weights. The environment's job is purely generating experience.

Practical Decision Framework

When building an RL environment for an LLM agent:

Do you have ground truth answers?

- Yes → Exact-match or code-execution verifier
- No → LLM-as-judge, checklist, or pairwise comparison

How many tool calls per episode?

- < 5 → Single-turn or simple tool-use env, no context management needed
- 5–50 → Multi-turn with basic context management
- 50–600 → Full agentic env with reference-preserving context management

Where should the model live?

- Experimenting across many environments → Model outside (Option A/C), use Prime Intellect Hub
- Tight RL training loop → Model co-located (Option B)
- No GPU access → Tinker API

What reward granularity?

- Simple tasks → Outcome-level
- Long-horizon tasks → Turn-level (Nanbeige4.1) or process rewards (PRIME)
- Open-ended research → Evolving rubrics (RLER) or checklist-style (Step-DR)

How to scale environments?

- Manual curation → High quality, expensive, this is where you start

- [AutoEnv](#) → Automated generation at ~\$4/env
- [AgentScaler](#) → Systematic scaling of heterogeneous simulated environments

Three things to pay attention to.

Environment diversity matters as much as environment quality. [AgentScaler](#)'s key finding is that heterogeneity of environments drives capability breadth in ways that simply adding more data from the same distribution cannot. You need more kinds of environments, not just more environments.

Automated environment generation is viable. At \$4 per generated environment, cost is no longer the bottleneck. The bottleneck is verifier quality — auto-generated environments with weak reward functions will teach the wrong behaviors at scale. ([AutoEnv](#))

The environment-as-package model is winning. The [Prime Intellect Environments Hub](#) is creating a shared ecosystem around RL environments, in the same way PyPI and HuggingFace created ecosystems around code and model weights. Environments published once, consumed by any trainer. This is a significant infrastructure shift.

The model isn't the only variable. The training ground shapes what the model can become. The task distribution, the harness, the verifier, the state management, the topology — these are the decisions that separate agents that work from agents that demo.

References

```
@article{
  leehanchung,
  author = {Lee, Hanchung},
  title = {The Training Grounds: A Taxonomy of RL Environments for},
  year = {2026},
  month = {03},
  day = {21},
  howpublished = {\url{https://leehanchung.github.io}},
  url = {https://leehanchung.github.io/blogs/2026/03/21/rl-environm
}
```

[Daily article] May 23: Running Out of Time (song)

ENGLISH WIKIPEDIA ARTICLE OF THE DAY · 23 MAY 2026 · [SOURCE](#)

"Running Out of Time" is a song by the American rock band Paramore (pictured). It was released on May 23, 2023, by Atlantic Records as the fourth single from the band's sixth studio album *This Is Why* (2023). Hayley Williams, Taylor York, and Zac Farro wrote the song in the early stages of the album's production, and Carlos de la Garza produced it. The band premiered the song at a concert at the Grand Ole Opry on February 7, 2023, and performed it on Jimmy Kimmel Live! on February 14. Described by critics as a pop-rock and dance-rock song, "Running Out of Time" deals with time management and comments on social anxieties over death, aging, and losing friends. Williams said the song was influenced by her personal struggle with punctuality and her friendship with Taylor Swift. Critics positively reviewed the song, praising its lyrics as playful and its composition as well-balanced. The song peaked on several international charts in 2023 and ranked on Billboard's year-end Alternative Airplay chart. (Full article...).

Read more: https://en.wikipedia.org/wiki/Running_Out_of_Time_%28song%29

Today's selected anniversaries:

1980:

The *Shining*, the psychological horror film directed by Stanley Kubrick, was premiered on 10 screens in New York City and Los Angeles. https://en.wikipedia.org/wiki/The_Shining_%28film%29

2008:

The International Court of Justice awarded the Middle Rocks to Malaysia and Pedra Branca to Singapore, resolving a 29-year-old territorial dispute in the Singapore Strait. https://en.wikipedia.org/wiki/Pedra_Branca_dispute

2021:

Ryanair Flight 4978 was forced to land by Belarusian authorities, who detained dissident journalist Roman Protasevich. https://en.wikipedia.org/wiki/Roman_Protasevich

Wiktionary's word of the day:

fervent: 1. Very hot; burning, scorching; also, glowing with heat. 2. (obsolete) 3. Of cold: intense, severe. 4. (pharmacy, historical) Of a drug: having an abundance of heat as one of the four qualities believed to be characteristic of all things; hot rather than cold, dry, or moist. 5. (figurative) 6. Of a person, or their actions, interests, or nature: exhibiting persistent belief or conviction, and/or enthusiasm or zeal; ardent, earnest, passionate. 7. (archaic) Of a thing such as a battle, unrest, etc.: intense, severe; also, of a wild animal: fierce. 8. About Word of the Day 9. Nominate a word 10. Leave feedback <https://en.wiktionary.org/wiki/fervent>

Wikiquote quote of the day:

There are addictions to feed and there are mouths to pay So you bargain with the Devil but you're okay for today. --Jewel (singer) https://en.wikiquote.org/wiki/Jewel_%28singer%29

2.1.150

CHANGELOG · 23 MAY 2026 · [SOURCE](#)

- Internal infrastructure improvements (no user-facing changes)
-

The Bag-of-Documents Model for Query Understanding and Retrieval

DANIEL TUNKELANG · 19 MAY 2026 · [SOURCE](#)

Today I had the pleasure of presenting my [recent work on the bag-of-documents model](#) as part of a series of talks hosted by [Trey Grainger](#) and [Doug Turnbull](#) in the context of their [AI-powered search classes](#).

You can find the [slides](#) below. I also encourage you to check out the [bag-of-documents Github repository](#), the [dataset and model on Hugging Face](#), as well as the live [Amazon demo](#) and [Best Buy demo](#). Enjoy!

<https://medium.com/media/12dd1c01b9dd78114555f2c6ed60a2fe/href>